

Langages formels

L3 Informatique / Math-Info ENS Paris-Saclay
2025-2026

Transparents élaborés par Paul Gastin et Stefan Schwoon

Rappel: Grammaires

Définition : $G = \langle \Sigma, V, P, S \rangle$

- ▶ Σ alphabet des *terminaux* (ensemble fini)
- ▶ V alphabet des *variables* (ensemble fini)
- ▶ P ensemble fini de *productions*
- ▶ S variable initiale

Hiérarchie de Chomsky (1956) :

- ▶ Grammaire de type 0 : $P \subseteq (\Sigma \cup V)^+ \times (\Sigma \cup V)^*$
- ▶ Grammaire de type 1 : $P \subseteq \{ \alpha \rightarrow \beta \mid \alpha, \beta \in (\Sigma \cup V)^+, |\alpha| \leq |\beta| \}$
- ▶ Grammaire de type 2 : $P \subseteq V \times (\Sigma \cup V)^+$
- ▶ Grammaire de type 3 : $P \subseteq V \times (\Sigma V \cup \Sigma)$

Exemple de type 2: $S \rightarrow aSb \mid ab$ engendre $\{ a^n b^n \mid n \geq 1 \}$.

Le cas du mot vide

Remarque : Exception $S \rightarrow \varepsilon$

Avec la définition précédente, les grammaires du type 0 sont les seules à générer le mot vide. Afin d'établir une meilleure correspondance entre grammaires et automates, on ajoute l'exception suivante pour les autres types :

La production $S \rightarrow \varepsilon$ est permise si S ne figure pas dans la partie droite d'une production.

Grammaire de type 2 faible : $P \subseteq V \times (\Sigma \cup V)^*$

Proposition : Elimination des productions ε

Soit $G = \langle \Sigma, V, P, S \rangle$ une grammaire de type 2 faible. Il existe une grammaire de type 2 qui engendre le même langage que G .

Du coup, nous allons utiliser la version faible de la définition.

Une grammaire de type 2 (ainsi que son langage engendré) s'appelle aussi *hors-contexte* ou *algébrique*.

Elimination des ε

Preuve

Soit $G' = \langle \Sigma, V \uplus \{S_0\}, P', S_0 \rangle$. On obtient P' en commençant par $P \cup \{S_0 \rightarrow S\}$ et en répétant la modification suivante :

- ▶ Tant que P' contient une production $A \rightarrow \varepsilon$, avec $A \neq S_0$, éliminer $A \rightarrow \varepsilon$ et, pour toute production $B \rightarrow \alpha A \beta$, ajouter $B \rightarrow \alpha \beta$.

Exemple : La grammaire faible $S \rightarrow aSb \mid \varepsilon$ engendre $\{a^n b^n \mid n \geq 0\}$.

Conversion en grammaire de type 2 :

- ▶ Ajouter $S_0 \rightarrow S$.
- ▶ Remplacer $S \rightarrow \varepsilon$ par $S_0 \rightarrow \varepsilon$ et $S \rightarrow ab$.

La grammaire résultante $(S_0 \rightarrow S \mid \varepsilon, S \rightarrow aSb \mid ab)$ est bien équivalente.

Dérivations / ambigüité

Définition : Dérivations

- ▶ dérivation gauche $\rightarrow_g^*$: remplacer toujours la variable la plus à gauche
- ▶ dérivation droite $\rightarrow_d^*$: analogue

Définition : Ambigüité

- ▶ Une grammaire est ambigüe s'il existe deux arbres de dérivations (distincts) de même racine et de même frontière.
- ▶ Un langage algébrique est *non ambigu* s'il existe une grammaire non ambigüe qui l'engendre. Dans le cas contraire, on dit qu'il est *inhéremment ambigu*.

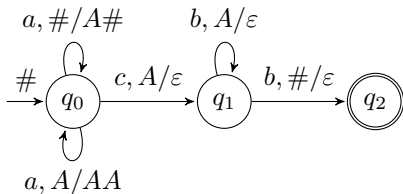
Exemple :

Arbres de dérivation pour $G_1 := S \rightarrow SS \mid aSb \mid \varepsilon$ et $G_2 := S \rightarrow aSbS \mid \varepsilon$.

Automate à pile (Exemple)

Exemple : Automate $\mathcal{A}_1$

On considère un espèce d'automate équipé d'une pile de symboles.



Les transitions portent deux annotations supplémentaires, notées A/w :

- ▶ A note le symbole qui doit être au sommet de la pile.
- ▶ La transition remplace A par w (un mot), le nouveau sommet de pile étant le premier symbole de w (sommet à gauche).
- ▶ La flèche qui marque l'état initial contient le contenu initial de la pile.

Automates à pile (AAP)

Définition : $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0z_0, F \rangle$ où

- ▶ Q ensemble fini d'états
- ▶ Σ alphabet d'entrée
- ▶ Z alphabet de pile
- ▶ $T \subseteq QZ \times (\Sigma \cup \{\varepsilon\}) \times QZ^*$ ensemble fini de transitions
- ▶ $q_0z_0 \in QZ$ configuration initiale
- ▶ $F \subseteq Q$ acceptation par état final.

Notation graphique : voir transparent précédent

Cas spéciaux :

- ▶ $\mathcal{A}$ est *temps-réel* (TR) s'il n'a pas d' ε -transition.
- ▶ $\mathcal{A}$ est *simple* (S) s'il ne possède qu'un seul état (que l'on ne note pas).

Sémantique d'un AAP

Définition : Système de transitions (infini) associé

- ▶ $\mathcal{T} = \langle QZ^*, T', q_0z_0, FZ^* \rangle$
- ▶ Une configuration de $\mathcal{A}$ est un élément $ph \in QZ^*$ de $\mathcal{T}$
- ▶ Transitions de $\mathcal{T}$: $T' = \{ pzh \xrightarrow{a} qgh \mid \langle pz, a, qg \rangle \in T, h \in Z^* \}$
- ▶ $\mathcal{L}(\mathcal{A}) = \{ w \in \Sigma^* \mid \exists qh \in FZ^* : q_0z_0 \xrightarrow{w}^* qh \}$

Exemple : $\mathcal{L}(\mathcal{A}_1) = \{ a^n cb^n \mid n \geq 1 \}$

Exemple de calcul:

$$q_0\# \xrightarrow{a} q_0A\# \xrightarrow{a} q_0AA\# \xrightarrow{c} q_1A\# \xrightarrow{b} q_1\# \xrightarrow{b} q_2$$

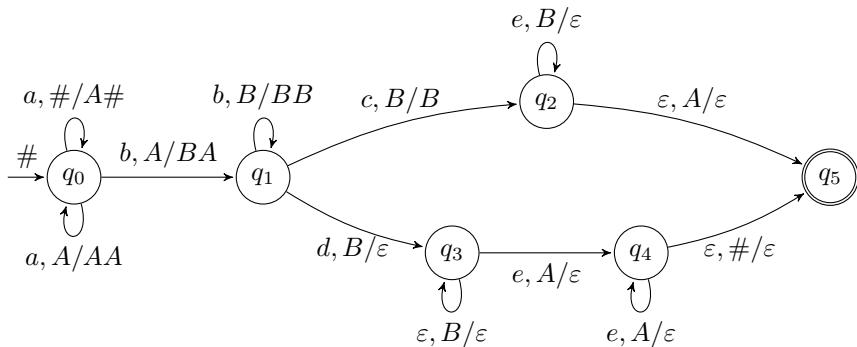
Remarque : On récupère les NFA avec $Z = \{z\}$ et z/z sur toute transition.

AAP avec ε -transitions

Attention, les ε -transitions peuvent s'enchaîner, en modifiant la pile :

Exemple : Automate $\mathcal{A}_2$

Cet automate accepte $\{a^n b^k c e^k \mid n, k \geq 1\} \cup \{a^n b^k d e^n \mid n, k \geq 1\}$.



Automates à pile: Exemples

Exemples :

- ▶ $L_1 = \{ a^n b^n c^k \mid n, k > 0 \}$ et $L_2 = \{ a^n b^k c^k \mid n, k > 0 \}$
- ▶ $L = L_1 \cup L_2$ (non déterministe)

Exercices :

1. Montrer que le langage $\{ w\tilde{w} \mid w \in \Sigma^* \}$ et son complémentaire peuvent être acceptés par un AAP. (Note: $\tilde{w}$ = miroir de w)
2. Montrer que le *complémentaire* du langage $\{ ww \mid w \in \Sigma^* \}$ peut être accepté par un AAP.
3. Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0, F \rangle$ un AAP. Montrer qu'on peut construire un AAP équivalent $\mathcal{A}'$ tel que $T' \subseteq Q'Z \times (\Sigma \cup \{\varepsilon\}) \times Q'Z^{\leq 2}$.
4. Même question pour les AAP temps-réel.

Motivation

Intérêt des AAP et grammaires algébriques :

Début dans les années 1950s :

- ▶ étude des grammaires formelles (connection linguistique/informatique)
- ▶ premiers ordinateurs : ajout d'une pile pour faciliter la programmation modulaire
- ▶ développement des langages de programmation : analyse syntaxique les machines de Turing

→ équivalences entre grammaires et modèles de calcul

Programme (pour aujourd'hui) :

- ▶ établir quelques propriétés fondamentales des AAP
- ▶ équivalence entre AAP et grammaires algébriques

Dans la suite, sauf mention particulière, toute *grammaire* est *algébrique*.

Acceptation généralisée

Définition :

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0 \rangle$ un AAP et $K \subseteq QZ^*$ un langage reconnaissable. Le langage reconnu par $\mathcal{A}$ avec *acceptation généralisée* K est

$$\mathcal{L}_K(\mathcal{A}) = \{ w \in \Sigma^* \mid \exists qh \in K : q_0 z_0 \xrightarrow{w}^* qh \}$$

Cas particuliers :

- ▶ $K = FZ^*$: acceptation classique **par état final**.
- ▶ $K = Q$: acceptation **par pile vide**.
- ▶ $K = F$: acceptation **par pile vide et état final**.
- ▶ $K = QZ'Z^*$ avec $Z' \subseteq Z$: acceptation **par sommet de pile**.

Exemple :

$\mathcal{L}(\mathcal{A}_1) = \{ a^n c b^n \mid n \geq 1 \}$ peut être accepté par pile vide ou par sommet de pile.

Proposition : Acceptation généralisée

Soit $\mathcal{A}$ un automate à pile avec acceptation généralisée K . On peut effectivement construire un automate à pile $\mathcal{A}'$ acceptant par état final tel que $\mathcal{L}_K(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$. Du coup, tous les modes d'acceptation ci-dessus sont équivalents.

Preuve

Idée : $\mathcal{A}'$ se comporte comme $\mathcal{A}$, mais à n'importe quel moment il peut basculer dans une phase de vérification qui simule un automate fini pour K .

Soit $\mathcal{A}_f = \langle Q \cup Z, S, \delta, s_0, F \rangle$ un NFA qui accepte K . Alors $\mathcal{A}' = \langle Q \uplus S, \Sigma, Z, T', q_0 z_0, F \rangle$ avec

$$\begin{aligned} T' = T \cup \{ \langle q, z, \varepsilon, s', z \rangle \mid \langle s_0, q, s' \rangle \in \delta, z \in Z \} \\ \cup \{ \langle s, z, \varepsilon, s', \varepsilon \rangle \mid \langle s, z, s' \rangle \in \delta \} \end{aligned}$$

Attention, cette vérification détruit la pile !

Automates à pile et grammaires

Proposition : Grammaire vers AAP

Soit $G = \langle \Sigma, V, P, S \rangle$ une grammaire. On peut construire un AAP *simple* $\mathcal{A}$ qui accepte $\mathcal{L}_G(S)$ par pile vide.

Preuve (dérivation gauche / top-down – idée)

$\mathcal{A} = \langle \Sigma, \Sigma \cup V, T, S \rangle$ avec deux types de transitions dans T :

- ▶ expansions : $\langle A, \varepsilon, \alpha \rangle$ pour tout $A \rightarrow \alpha \in P$,
- ▶ vérifications : $\langle a, a, \varepsilon \rangle$ pour tout $a \in \Sigma$.

Pour tout $\alpha \in (\Sigma \cup V)^*$ et $w \in \Sigma^*$, prouver $\alpha \rightarrow_g^* w$ dans G ssi $\alpha \xrightarrow{w}^* \varepsilon$ dans $\mathcal{A}$:

- ▶ $\Rightarrow$: récurrence sur longueur de dérivation
- ▶ $\Leftarrow$: récurrence sur longueur du calcul

Automates à pile et grammaires

Proposition : Automate à pile vers grammaire

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0 \rangle$ un automate à pile reconnaissant par pile vide. On peut construire une grammaire G qui engendre $\mathcal{L}(\mathcal{A})$.

Construction:

- ▶ $G = \langle \Sigma, \{S\} \cup (Q \times Z \times Q), P, S \rangle$
- ▶ pour tout $q \in Q$, $S \rightarrow \langle q_0, z_0, q \rangle \in P$;
- ▶ pour tout $a \in \Sigma \cup \{\varepsilon\}$ et $\langle qz, a, q' \rangle \in T$, $\langle q, z, q' \rangle \rightarrow a \in P$;
- ▶ pour tout $a \in \Sigma \cup \{\varepsilon\}$, $n \geq 1$, $\langle qz, a, q' z_1 \cdots z_n \rangle \in T$ et $q_1, \dots, q_n \in Q$,

$$\langle q, z, q_n \rangle \rightarrow a \langle q', z_1, q_1 \rangle \langle q_1, z_2, q_2 \rangle \cdots \langle q_{n-1}, z_n, q_n \rangle \in P$$

Preuve (idée): $\langle q, z, q' \rangle \rightarrow^* w$ dans G ssi $qz \xrightarrow{w}^* q'$ dans $\mathcal{A}$
(par récurrence sur longueur de dérivation resp. calcul)

Forme normale de Chomsky

Définition : FNC

Soit $G = (\Sigma, V, P, S)$ une grammaire. G est dite en *forme normale de Chomsky* (FNC) si $P \subseteq (V \times (V^2 \cup \Sigma)) \cup \{S \rightarrow \varepsilon\}$; autrement dit, toute production est de la forme (i) $A \rightarrow BC$, (ii) $A \rightarrow a$ ou (iii) $S \rightarrow \varepsilon$.

Si $S \rightarrow \varepsilon \in P$, alors $B \neq S$ et $C \neq S$ dans les productions de type (i).

Théorème : Conversion en FNC

Pour toute grammaire G il existe une grammaire FNC G' telle que $\mathcal{L}(G) = \mathcal{L}(G')$.

Preuve (idée):

- ▶ Introduire une nouvelle variable initiale S_0 et $S_0 \rightarrow S$.
- ▶ Pour tout $a \in \Sigma$, introduire variable V_a ; remplacer toute occurrence de a dans les productions par V_a et ajouter $V_a \rightarrow a$.
- ▶ Limiter la longueur de la côte droite de toute production à deux (p.ex. remplacer $A \rightarrow BCD$ par $A \rightarrow C'D$ et $C' \rightarrow BC$).
- ▶ Éliminer toutes les productions du type $B \rightarrow \varepsilon$ (sauf $S_0 \rightarrow \varepsilon$).
- ▶ Pour toute paire $A \rightarrow B$ et $B \rightarrow \beta$, ajouter $A \rightarrow \beta$.
- ▶ Supprimer toute production de la forme $A \rightarrow B$.

Lemme d'itération

Théorème : Bar-Hillel, Perles, Shamir ou Lemme d'itération

Soit $L \subseteq \Sigma^*$ algébrique. Il existe $N \geq 0$ tel que pour tout $w \in L$, si $|w| \geq N$ alors on peut trouver une factorisation $w = \alpha u \beta v \gamma$ avec (i) $|uv| > 0$ et (ii) $|u \beta v| \leq N$ et (iii) $\alpha u^n \beta v^n \gamma \in L$ pour tout $n \geq 0$.

Preuve: Soit G une grammaire FNC avec k variables et $\mathcal{L}(G) = L$. Choisir $N := 2^k$. Un arbre de dérivation de n'importe quel mot $w \in L$ avec $|w| \geq N$ contient un chemin avec au moins $k + 1$ variables. Parmi les $k + 1$ dernières variables d'un chemin de longueur maximale, une variable A se répète, du coup:

$$S \rightarrow^* \alpha A \gamma \rightarrow^+ \alpha u A v \gamma \rightarrow^+ \alpha u \beta v \gamma$$

avec les propriétés (i), (ii) et (iii).

Exemple :

Le langage $L_1 = \{a^n b^n c^n \mid n \geq 0\}$ n'est pas algébrique.

Corollaire :

Le langages algébriques ne sont pas fermées par intersection ou complémentaire.

Clôture et réduction

Soit Γ un alphabet, $\bar{\Gamma} = \{\bar{a} \mid a \in \Gamma\}$ une copie de Γ et $\tilde{\Gamma} = \Gamma \uplus \bar{\Gamma}$.

On définit la réduction sur $\tilde{\Gamma}^*$ par $\bar{a}a \xrightarrow{\text{red}} \varepsilon$ pour $a \in \Gamma$.

Pour $L \subseteq \tilde{\Gamma}^*$ on pose $\text{Clot}(L) = \{w \in \tilde{\Gamma}^* \mid \exists v \in L, v \xrightarrow[*]{\text{red}} w\}$.

Lemme : Reconnaissabilité de la clôture

Si $L \subseteq \tilde{\Gamma}^*$ est un langage reconnaissable alors $\text{Clot}(L) \subseteq \tilde{\Gamma}^*$ aussi.

On peut construire un automate pour $\text{Clot}(L)$ à partir d'un automate pour L .

Preuve Procédure de saturation (PTIME)

Itérativement ajouter des transitions $p \xrightarrow{\varepsilon} q$ pour tout triplet p, q, a tel que $p \xrightarrow[*]{\bar{a}a} q$.

Configurations accessibles

Proposition : Reconnaissabilité des configurations accessibles

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0 \rangle$ un automate à pile.

Soit $L \subseteq QZ^*$ un langage reconnaissable, on note

$$\mathcal{C}(L) = \{qh \in QZ^* \mid \exists pg \in L : pg \rightarrow_{\mathcal{A}}^+ qh\}$$

l'ensemble des configurations accessibles à partir de celles de L .

On peut effectivement construire un automate fini $\mathcal{B}$ qui reconnaît $\mathcal{C}(L)$.

On définit $\Gamma = Q \uplus Z$ et le langage fini $K = \{qh\bar{x}\bar{p} \mid \langle p, x, a, q, h \rangle \in T\} \subseteq \tilde{\Gamma}^+$.

Lemme :

Soit $n \geq 0$. Il existe un calcul $pg \xrightarrow{n} qh$ dans $\mathcal{A}$ ssi il existe $w \in K^n$ t.q. $wpg \xrightarrow{\text{red}}_{2n} qh$

Corollaire : $\mathcal{C}(L) = \text{Clot}(K^+L) \cap QZ^*$.

Puisque K est un langage fini, le langage K^+L est reconnaissable et on peut construire (P_{TIME}) un automate $\mathcal{B}$ qui reconnaît $\text{Clot}(K^+L) \cap QZ^*$.

Par analogie, $\bar{\mathcal{C}}(L) = \{qh \in QZ^* \mid \exists pg \in L : qh \rightarrow_{\mathcal{A}}^+ pg\}$ est reconnaissable.

Calculs d'accessibilité

Corollaire :

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0 \rangle$ un automate à pile.

On peut effectivement calculer les ensembles suivants :

1. $X = \{ \langle p, x, q \rangle \in Q \times Z \times Q \mid px \rightarrow_{\mathcal{A}}^+ q \}$
2. $Y = \{ \langle p, x, q, y \rangle \in Q \times Z \times Q \times Z \mid px \rightarrow_{\mathcal{A}}^+ qy \}$
3. $W = \{ \langle p, x, q, y \rangle \in Q \times Z \times Q \times Z \mid \exists h : px \rightarrow_{\mathcal{A}}^+ qyh \}$
4. $X' = \{ \langle p, x, q \rangle \in Q \times Z \times Q \mid px \xrightarrow{\varepsilon}_{\mathcal{A}}^+ q \}$
5. $Y' = \{ \langle p, x, q, y \rangle \in Q \times Z \times Q \times Z \mid px \xrightarrow{\varepsilon}_{\mathcal{A}}^+ qy \}$
6. $W' = \{ \langle p, x, q, y \rangle \in Q \times Z \times Q \times Z \mid \exists h : px \xrightarrow{\varepsilon}_{\mathcal{A}}^+ qyh \}$

Preuve

1. $\langle p, x, q \rangle \in X$ ssi $q \in \mathcal{C}(px)$.
2. $\langle p, x, q, y \rangle \in Y$ ssi $qy \in \mathcal{C}(px)$.
3. $\langle p, x, q, y \rangle \in W$ ssi $\mathcal{C}(px) \cap qyZ^* \neq \emptyset$.

On applique ce qui précède à l'automate $\mathcal{A}'$ obtenu à partir de $\mathcal{A}$ en ne conservant que les ε -transitions.

Existence d'un livelock

Proposition :

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0 \rangle$ un AAP et $p\alpha \in QZ^*$. On peut décider s'il existe une suite infinie de transitions ε depuis $p\alpha$ (noté $p\alpha \xrightarrow{\varepsilon}_{\mathcal{A}}^{\omega}$).

Lemme :

$p\alpha \xrightarrow{\varepsilon}_{\mathcal{A}}^{\omega}$ ssi $\exists q \in Q, z \in Z$ et $g, h \in Z^*$ tels que (i) $p\alpha \xrightarrow{\varepsilon}_{\mathcal{A}}^* qzg$ et (ii) $qz \xrightarrow{\varepsilon}_{\mathcal{A}}^+ qzh$.

Preuve

Sens indirect : immédiat

Sens direct : Soit $(p_i z_i \alpha_i)_{i \geq 0}$ la suite des configurations dans le livelock. On choisit $i_1 < i_2 < \dots$ tq $\forall j \forall k > i_j : |\alpha_{i_j}| \leq |\alpha_k|$. Par principe de tiroirs il existe qz tq $|\{j \mid p_{i_j} = q \wedge z_{i_j} = z\}| = \infty$. Soient $j_1 < j_2$ deux indices dans cet ensemble, alors $g := \alpha_{i_{j_1}}$ et h tq $\alpha_{i_{j_2}} = hg$.

On obtient la proposition en identifiant tous les qz qui satisfont la condition (ii) du lemme, et en appliquant les calculs du transparent précédent.

Calculs d'accessibilité dans une grammaire

Proposition : Reconnaissabilité des configurations précédentes

Soit $G = \langle \Sigma, V, \rightarrow, S \rangle$ une grammaire algébrique.

Soit $L \subseteq (\Sigma \cup V)^*$ reconnaissable (par un automate fini $\mathcal{A}$).

On note $pre(L) = \{ \alpha \in (\Sigma \cup V)^* \mid \exists \beta \in L : \alpha \rightarrow_G^* \beta \}$.

On peut construire (en PTIME) un automate fini $\mathcal{B}$ qui reconnaît $pre(L)$.

Preuve : Voir transparent suivant.

Corollaire : Corollaires

Les problèmes suivants sont décidables en PTIME:

- ▶ mot: pour $w \in \Sigma$, $w \in \mathcal{L}(G)$?
- ▶ variable productive: pour $A \in V$, existe-il $w \in \Sigma^*$ t.q. $A \rightarrow_G^* w$?
- ▶ $\mathcal{L}(G)$ est-il fini ?

Automate pour $pre(L)$

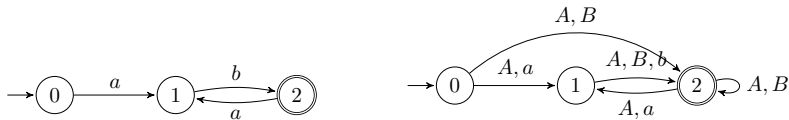
Proposition :

Soit $\mathcal{A} = \langle Q, \Sigma \cup V, \delta, q_0, F \rangle$ un automate reconnaissant L .

Construire $\mathcal{B}$ en ajoutant (itérativement) des transitions selon la règle suivante :

Si $\langle A, \beta \rangle \in \rightarrow$ et $q \xrightarrow{\beta}_{\mathcal{B}} q'$, ajouter $\langle q, A, q' \rangle$ dans $\mathcal{B}$.

Exemple: $A \rightarrow a \mid BB, \quad B \rightarrow AB \mid b$



Preuve (idée): $L(\mathcal{B}) = pre(\mathcal{L}(\mathcal{A}))$

- ▶ $\subseteq$: récurrence sur nombre de transitions ajoutées
- ▶ $\supseteq$: récurrence sur longueur de dérivation

Langages déterministes

Définition : Automate à pile déterministe (AAPD)

$\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0, F \rangle$ est *déterministe* si

- ▶ $\forall (pz, a) \in QZ \times (\Sigma \cup \{\varepsilon\}), \quad |T(pz, a)| \leq 1,$
- ▶ $\forall pz \in QZ, \quad T(pz, \varepsilon) \neq \emptyset \implies \forall a \in \Sigma : T(pz, a) = \emptyset$

Un langage $L \subseteq \Sigma^*$ est *déterministe* s'il existe un AAPD qui accepte L par état final.

Théorème : Les déterministes sont fermés par complémentaire.

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0, F \rangle$ un AAPD, on peut effectivement construire un AAPD $\mathcal{A}'$ qui reconnaît $\Sigma^* \setminus \mathcal{L}(\mathcal{A})$.

Il y a deux difficultés principales :

1. Un automate déterministe peut se bloquer (deadlock) ou entrer dans un ε -calcul infini (livelock). Dans ce cas il y a des mots qui n'admettent aucun calcul dans l'automate.
2. Même avec un automate déterministe, un mot peut avoir plusieurs calculs (ε -transitions à la fin), certains réussis et d'autres non.

Blocage

Définition : Blocage

Un AAPD $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0 \rangle$ est *sans blocage* si pour toute configuration accessible $p\alpha$ et pour toute lettre $a \in \Sigma$ il existe un calcul $p\alpha \xrightarrow{\varepsilon}^* \xrightarrow{a}$.

Proposition : Critère d'absence de blocage

Un AAPD est sans blocage ssi pour toute configuration accessible $p\alpha$ on a

1. $\alpha \neq \varepsilon$, on peut donc écrire $\alpha = z\beta$ avec $z \in Z$,
2. $T(pz, \varepsilon) \neq \emptyset$ ou $\forall a \in \Sigma : T(pz, a) \neq \emptyset$
3. il n'y a pas de livelock ($pz \xrightarrow{\varepsilon}^\omega$)

De plus, ces critères sont décidables.

Remarque :

Si $\mathcal{A}$ est sans blocage alors chaque mot $w \in \Sigma^*$ a un unique calcul maximal (et fini) $q_0 z_0 \xrightarrow{w}^* pz\alpha$ dans $\mathcal{A}$ et $T(pz, \varepsilon) = \emptyset$.

Suppression des blocages

Proposition : Suppression des blocages

Soit $\mathcal{A} = (Q, \Sigma, Z, T, q_0 z_0, F)$ un automate à pile déterministe, on peut effectivement construire un automate à pile déterministe *sans blocage* $\mathcal{A}' = (Q', \Sigma, Z', T', q'_0 z'_0, F')$ qui reconnaît le même langage.

Preuve

$Q' = Q \uplus \{q'_0, d, f\}$, $F' = F \uplus \{f\}$, $Z' = Z \uplus \{\perp\}$, $z'_0 = \perp$ et pour $p \in Q$, $a \in \Sigma$ et $z \in Z$

1. Marqueur fin de pile: $q'_0 \perp \xrightarrow{\varepsilon} q_0 z_0 \perp$, $p \perp \xrightarrow{\varepsilon} d \perp$, $d \perp \xrightarrow{a} d \perp$ dans T'
2. Garder les transitions non- ε : Si $pz \xrightarrow{a} q\alpha \in T$ alors $pz \xrightarrow{a} q\alpha \in T'$.
3. Garder les ε hors livelock: Si $pz \xrightarrow{\varepsilon} q\alpha \in T$ et $\neg(pz \xrightarrow{\varepsilon}^\omega_{\mathcal{A}})$, alors $pz \xrightarrow{\varepsilon} q\alpha \in T'$.
4. Traiter les livelock: Si $pz \xrightarrow{\varepsilon}^\omega_{\mathcal{A}}$ alors $pz \xrightarrow{\varepsilon} fz \in T'$, s'il existe $q' \in F$ avec $pz \xrightarrow{\varepsilon}^*_{\mathcal{A}} q'\alpha$, $pz \xrightarrow{\varepsilon} dz \in T'$ sinon.
5. f n'accepte que mot vide: $fz \xrightarrow{a} dz$, $dz \xrightarrow{a} dz$ dans T'
6. Si $T(pz, a) = \emptyset$ et $T(pz, \varepsilon) = \emptyset$, alors $pz \xrightarrow{a} dz \in T'$.

Complémentaire

Proposition :

Soit $\mathcal{A} = (Q, \Sigma, Z, T, q_0 z_0, F)$ un automate à pile déterministe, on peut effectivement construire un automate à pile déterministe $\mathcal{A}'$ qui reconnaît $\Sigma^* \setminus \mathcal{L}(\mathcal{A})$.

Proposition :

Soit $\mathcal{A} = (Q, \Sigma, Z, T, q_0 z_0, F)$ un automate à pile déterministe, on peut effectivement construire un automate à pile déterministe équivalent $\mathcal{A}'$ tel qu'on ne puisse pas faire d' ε -transition à partir d'un état final de $\mathcal{A}'$.

Complémentaire (construction)

Preuve : Complémentaire

On suppose $\mathcal{A}$ déterministe et sans blocage. On pose $Q' = Q \times \{1, 2, 3, 4\}$. L'état initial est $q'_0 = \langle q_0, 1 \rangle$ si $q_0 \notin F$ et $q'_0 = \langle q_0, 2 \rangle$ sinon.

1. Si $pz \xrightarrow{\varepsilon} q\alpha \in T$ et $i \in \{1, 2\}$ alors
$$\langle p, i \rangle z \xrightarrow{\varepsilon} \langle q, j \rangle \alpha \in T' \text{ avec } j = \begin{cases} 1 & \text{si } i = 1 \wedge q \notin F \\ 2 & \text{sinon.} \end{cases}$$
2. Si $pz \xrightarrow{a} q\alpha \in T$ et $i \in \{1, 2\}$ alors $\langle p, i \rangle z \xrightarrow{\varepsilon} \langle p, i + 2 \rangle z \in T'$ et
$$\langle p, i + 2 \rangle z \xrightarrow{a} \langle q, j \rangle \alpha \in T' \text{ avec } j = \begin{cases} 1 & \text{si } q \notin F \\ 2 & \text{sinon.} \end{cases}$$

L'automate $\mathcal{A}'$ lui aussi est déterministe et sans blocage.

Soit $w \in \Sigma^*$ et $q_0 z_0 \xrightarrow{w}^* pz\alpha$ l'unique calcul *maximal* de $\mathcal{A}$ sur w .

L'unique calcul *maximal* de $\mathcal{A}'$ sur w s'écrit $q'_0 z_0 \xrightarrow{w}^* \langle p, j \rangle z\alpha$ avec $j = 3$ si le calcul de $\mathcal{A}$ n'a pas visité F depuis la dernière transition non- ε , et $j = 4$ sinon.

- ▶ Avec $F' = Q \times \{3\}$ on obtient $\mathcal{L}(\mathcal{A}') = \overline{\mathcal{L}(\mathcal{A})}$.
- ▶ Avec $F' = Q \times \{4\}$ on obtient $\mathcal{L}(\mathcal{A}') = \mathcal{L}(\mathcal{A})$.

Dans les deux cas, à partir d'un état de F' il n'y a pas d' ε -transition.

De plus, chaque mot $w \in \mathcal{L}(\mathcal{A}')$ a un unique calcul acceptant dans $\mathcal{A}'$.

Résumé: Propriétés de clôture

Proposition :

Les langages algébriques :

- ▶ sont clôturés par union ;
- ▶ ne sont pas clôturés par complément (voir $\{ ww \mid w \in \Sigma^* \}$) ni intersection.

Proposition :

Les langages déterministes :

- ▶ sont clôturés par complément ;
- ▶ ne sont pas clôturés par intersection (voir $\{ a^n b^p c^n \mid n, p \geq 1 \}$, $\{ a^n b^n c^p \mid n, p \geq 1 \}$) ni union ;

Hiérarchie de langages

Exercice :

1. Montrer que tout langage déterministe est non ambigu.
2. Montrer que le langage $L_1 = \{a^n b^n \mid n > 0\} \cup \{a^n b^{2n} \mid n > 0\}$ est non ambigu mais pas déterministe.
3. Montrer que le langage $L_2 = \{a^n b^p c^n \mid n, p > 0\} \cup \{a^n b^p d^p \mid n, p > 0\}$ est déterministe mais pas déterministe temps-réel (D+TR).

Remarque : Hiérarchie

- ▶ Les langages reconnaissables sont strictement inclus dans les D+TR.
- ▶ Les D+TR sont strictement inclus dans les déterministes.
- ▶ Les déterministes sont strictement inclus dans les langages non-ambigus.
- ▶ Les non-ambigus sont strictement inclus dans les algébriques.

AAP régulier

Remarques: Dans ce transparent et le suivant seulement:

- ▶ Σ' note $\Sigma \cup \{\varepsilon\}$;
- ▶ l'état et sommet de pile sont notés à droite !

Définition : AAP régulier

$\mathcal{A} = \langle Q, \Sigma, Z, T, q_0, F \rangle$, avec $|T|$ fini et

$$T \subseteq (\text{Rec}(Z^*) \times Q \times \Sigma' \times Z \times Q) \cup (\text{Rec}(Z^*) \times Q \times \Sigma' \times Q)$$

1. $wq \xrightarrow{a} wzq'$ si $\langle L, q, a, z, q' \rangle \in T$ et $w \in L$ (push)
2. $wzq \xrightarrow{a} wq'$ si $\langle L, q, a, q' \rangle \in T$ et $wz \in L$ (pop)

AAP régulier $\rightarrow$ AAP ordinaire

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0, F \rangle$ un AAP régulier avec $k := |T|$
et $\forall i : \mathcal{A}_i = \langle Q_i, Z, \delta_i, \iota_i, F_i \rangle$ DCA acceptant les langages dans T . Définissons:

- ▶ $\mathcal{Q} := Q_1 \times \dots \times Q_k, \quad \iota := \langle \iota_1, \dots, \iota_k \rangle$
- ▶ $\mathcal{F}_i := \{ \langle q_1, \dots, q_k \rangle \in \mathcal{Q} \mid q_i \in F_i \}$
- ▶ $\delta : \mathcal{Q} \times Z \rightarrow \mathcal{Q}$ avec $\delta(\langle q_1, \dots, q_k \rangle, z) := \langle \delta_1(q_1, z), \dots, \delta_k(q_k, z) \rangle$.

Construction d'un AAP ordinaire équivalent à $\mathcal{A}$

$\mathcal{A}' := \langle \mathcal{Q} \times \mathcal{Q}, \Sigma, \mathcal{Q} \times Z, T', \langle \iota, q_0 \rangle, \mathcal{Q} \times F \rangle$, avec:

- ▶ (push) pour tout $\langle L_i, q, a, z, q' \rangle \in T$ et $f \in \mathcal{F}_i$, on a $\langle \langle f, q \rangle, a, \langle f, z \rangle, \langle \delta(f, z), q' \rangle \rangle \in T'$;
- ▶ (pop) pour tout $\langle L_i, q, a, q' \rangle \in T, z \in Z, q'' \in \mathcal{Q}$ et $f \in F_i$, on a $\langle \langle q'', z \rangle, \langle f, q \rangle, a, \langle q'', q' \rangle \rangle \in T'$.

Invariante: Si $\mathcal{A}$ accède à une configuration $z_1 \dots z_n q$, alors $\mathcal{A}'$ accède à $\langle q'_0, z_1 \rangle \dots \langle q'_{n-1}, z_n \rangle \langle q'_n, q \rangle$, avec $q'_0 = \iota$ et $q'_{i+1} = \delta(q'_i, z_{i+1})$ pour $i = 0, \dots, n-1$.

Déterministes temps-réel

Proposition :

Les langages déterministes temps-réel sont strictement inclus dans les déterministes.

Preuve adapté d'[Igarashi 1985]

On considère le langage $L = \{a^n b^k c^n \mid n, k \geq 1\} \cup \{a^n b^k d^k \mid n, k \geq 1\}$ qui est déterministe. Supposons qu'il existe un AAP D+TR $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0, F \rangle$ qui accepte L . S.p.d.g., empile au plus deux symboles par transition. Soit $\Phi(n)$ le nombre de configurations de $\mathcal{A}$ avec au plus n symboles de pile, plus 1. On pose $F := |Q| \cdot |Z| + 1$, $N := \Phi(F)$, $M := \Phi(F + N)$.

Soit $C_{i,j}$ la configuration atteinte par $a^i b^j$ dans $\mathcal{A}$.

- ▶ $C_{i,j} = C_{i',j'}$ implique $i = i'$ et $j = j'$.
- ▶ Pour $i \leq N$, soit $j_i \leq M$ tq $|C_{i,j_i}| > F + N$.
- ▶ Soit i^* tq $\min\{|C_{i^*,j}| \mid j \leq M\} > F$.
- ▶ Pour $h \leq F$, soit $i_h = \max\{i < i^* \mid |C_{i,0}| = h\}$ (suite croissante).

Preuve (suite)

Il existe donc $q, q' \in Q, z \in Z, w_1, w_2, w_3 \in Z^*$ et $h_1 < h_2 \leq F$ tq

$$C_{0,0} \xrightarrow{a^{p_1}}^* qzw_1 \quad qz \xrightarrow{a^{p_2}}^* qzw_2 \quad qz \xrightarrow{a^{p_3} b^{j_{i^*}}}^* q'w_3$$

avec $p_1 = i_{h_1}, p_2 = i_{h_2} - p_1, p_3 = i^* - p_2$.

Rappel: $C_{i^*, j_{i^*}} = q'w_3w_2w_1$ et $|w_1w_2w_3| > F + N$, et $|w_1w_2| \leq F$.

Du coup, pour la configuration acceptante C^* atteinte $a^{i^*} b^{j_{i^*}} c^{i^*}$ on a $|C^*| > F$, et il existe un calcul $q'w_3 \xrightarrow{c^{i^*}}^* q_fw_4$, avec $q_f \in F, w_4 \in Z^*$.

Ainsi, le mot $w := a^{p_1+p_3} b^{j_{i^*}} c^{i^*}$ est accepté avec la configuration $q_fw_4w_1$, alors que $w \notin L$; contradiction.

Problèmes indécidables

Proposition :

Soient L, L' deux langages algébriques et R un langage rationnel.
Les problèmes suivants sont indécidables :

- ▶ $L \cap L' = \emptyset$?
- ▶ $L = \Sigma^*$?
- ▶ $L = L'$?
- ▶ $L \subseteq L'$?
- ▶ $R \subseteq L$?
- ▶ L est-il ambigu ?
- ▶ L est-il rationnel ?
- ▶ L est-il déterministe ?
- ▶ $\overline{L}$ est-il algébrique ?
- ▶ $L \cap L'$ est-il algébrique ?

Langages déterministes

Proposition : Décidabilité et indécidabilité

On ne peut pas décider si un langage algébrique est déterministe.

Soient L, L' deux langages déterministes et R un langage rationnel.

Les problèmes suivants sont décidables :

▶ $R \subseteq L$?

$$R \subseteq L \iff R \cap \overline{L} = \emptyset$$

▶ $L = R$?

$$R = L \iff R \cap \overline{L} = \emptyset = \overline{R} \cap L$$

▶ L est-il rationnel ?

▶ $L = L'$?

Géraud Sénizergues, prix Gödel 2002

Les problèmes suivants sont indécidables :

▶ $L \cap L' = \emptyset$?

▶ $L \subseteq L'$?

▶ $L \cap L'$ est-il algébrique ?

▶ $L \cap L'$ est-il déterministe ?

▶ $L \cup L'$ est-il déterministe ?

Réduction par PCP

Proposition : Rappel: PCP

Soient $f, g : A^* \rightarrow B^*$ deux morphismes. Le problème de correspondance de Post (PCP) est de décider s'il existe $w \in A^+$ tel que $f(w) = g(w)$. Ce problème est indécidable.

Lemme : Grammaire pour f

Le langage $L_f = \{ \tilde{v} \# f(v) \mid v \in A^+ \}$ est linéaire et déterministe.

Preuve

La grammaire G_f avec $F \rightarrow aFf(a)$ ($a \in A$), $F \rightarrow \#$ convient.

Corollaire :

1. Le vide de l'intersection de deux langages algébriques est indécidable.
2. L'ambiguïté d'une grammaire algébrique est indécidable.

Preuve

1. $\text{PCP}(f, g)$ a une solution ssi $L_f \cap L_g \neq \emptyset$.
2. G_f, G_g sont non-ambigues. On considère la grammaire G avec $S \rightarrow F \mid G$ comme symbole de départ. G est ambiguë ssi $\text{PCP}(f, g)$ a une solution.

Universalité

Lemme :

Le langage $L_{\neg f} = \{ \tilde{u}\#v \mid u \in A^+, v \in B^*, v \neq f(u) \}$ est linéaire et déterministe.

Preuve

La grammaire $G_{\neg f}$ suivante convient :

$$S \rightarrow aSf(a) \mid aS_1w_1 \mid aS_2w_2 \mid S_3$$

avec $a \in A$ et

- ▶ $\mathcal{L}(S_1) = A^*\#B^*$ et $|w_1| \leq |f(a)|$ mais pas suffixe de $f(a)$,
- ▶ $\mathcal{L}(S_2) = A^*\#$ et $|w_2| < |f(a)|$,
- ▶ $\mathcal{L}(S_3) = \#B^+$.

Corollaire :

L'universalité d'un langage linéaire est indécidable.

Preuve

Le langage $L = L_{\neg f} \cup L_{\neg g} \cup (A^+\#B^*)^c$ est linéaire.

$L = \Sigma^*$ (avec $\Sigma = A \cup B \cup \{\#\}$) ssi $\text{PCP}(f, g)$ n'a pas de solution.

Automates à pile visible

Définition : Alphabet visible

Un *alphabet visible* est un triplet d'alphabets $\Sigma = \langle \Sigma_2, \Sigma_1, \Sigma_0 \rangle$.

Dans la suite, Σ note un tel alphabet visible.

Définition : AAP visible

$A = \langle Q, \Sigma, Z, T_0 \cup T_1 \cup T_2, q_0 z_0, F \rangle$ où

$$T_0 \subseteq Q \times \Sigma_0 \times Z \times Q \quad T_1 \subseteq Q \times \Sigma_1 \times Q \quad T_2 \subseteq Q \times \Sigma_2 \times Q \times Z$$

Intuition:

- ▶ Avec Σ_0 on dépile un symbole, avec Σ_2 on en empile un.
- ▶ Les symboles de Σ_1 ne changent rien dans la pile.
- ▶ Ainsi, un mot w en entrée détermine la hauteur de la pile atteinte en lisant w .

Exemples:

- ▶ $L_1 = \{a^n b^n c^p \mid n, p > 0\}$ est accepté par un AAPV avec $\Sigma = \langle \{a\}, \{c\}, \{b\} \rangle$;
- ▶ $L_2 = \{a^n b^p c^p \mid n, p > 0\}$ est accepté par un AAPV avec $\Sigma = \langle \{b\}, \{a\}, \{c\} \rangle$.
- ▶ $L_1 \cup L_2$ est bien algébrique mais pas visible.

Clôture des langages visibles

Clôture

Les langages acceptés par les AAPV (langages visibles), pour un alphabet visible donné, sont fermés par union, concaténation, intersection et complément.

- ▶ Union: trivial
- ▶ Concaténation: comme pour automates fini, en supposant que les alphabets de pile sont disjoints.
- ▶ Intersection: par constructure d'un produit qui synchronise à la fois les états et les symboles de pile de deux VPA donnés.
- ▶ Complément: par déterminisation (voir la suite)

Déterminisation des AAPV

Déterminisation des AAPV

Soit $\mathcal{A} = \langle Q, \Sigma, Z, T, q_0 z_0, F \rangle$ un AAPV qui accepte le langage L .
Il existe un AAPV $\mathcal{A}'$ déterministe qui accepte L .

On suppose que z_0 figure uniquement en fin de pile et ne disparaît jamais.

On construit $\mathcal{A}' = \langle Q', \Sigma, Z', T_0 \cup T_1 \cup T_2, q'_0 z_0, F' \rangle$ ainsi :

- ▶ $Q' = 2^Q \times 2^{Q \times Q}$, $q'_0 = \langle \{q_0\}, \{ \langle q, q \rangle \mid q \in Q \} \rangle$, $F' = \{ \langle R, S \rangle \mid R \cap F \neq \emptyset \}$
- ▶ $Z' = (Q' \times \Sigma_2) \cup \{z_0\}$
- ▶ $T'_1 = \{ \langle \langle R, S \rangle, a, \langle R', S' \rangle \rangle \mid R' = \{ q' \mid \exists q \in R : \langle q, a, q' \rangle \in T_1 \}, S' = \{ \langle q, q'' \rangle \mid \exists q' : \langle q, q' \rangle \in S, \langle q', a', q'' \rangle \in T_1 \} \}$
- ▶ $T'_2 = \{ \langle \langle R, S \rangle, a, \langle R', S' \rangle, \langle R, S, a \rangle \rangle \mid R' = \{ q' \mid \exists q \in R : \langle q, a, q', z \rangle \in T_2 \}, S' = \{ \langle q, q \rangle \mid q \in Q \} \}$
- ▶ $T'_0 = \{ \langle \langle R, S \rangle, a, \langle R', S', a \rangle, \langle R'', S'' \rangle \rangle \mid R' = \{ q' \mid \exists q \in R' : \langle q, q' \rangle \in U \}, S' = \{ \langle q, q' \rangle \mid \exists q'' : \langle q, q'' \rangle \in S', \langle q'', q' \rangle \in U \} \}$
avec $U := \{ \langle q, q' \rangle \mid \exists a', q_1, q_2, z : \langle q, a', q_1, z \rangle \in T_2, \langle q_1, q_2 \rangle \in S, \langle q_2, a, z, q' \rangle \in T_0 \}$

Fonctions séquentielles

Définition : Automates séquentiels et fonctions séquentielles

$A = \langle Q, A, B, q_0, \delta, \otimes, m, \rho \rangle$ où

- ▶ Q est un ensemble fini d'états ;
- ▶ A et B sont les alphabets d'entrée et de sortie,
- ▶ $\delta : Q \times A \rightarrow Q$ est une fonction *partielle* de transition;
- ▶ $\otimes : Q \times A \rightarrow B^*$ fonction *partielle* de sortie avec $\text{dom}(\otimes) = \text{dom}(\delta)$;
- ▶ $q_0 \in Q$ est l'état initial et $m \in B^*$ est le préfixe initial ;
- ▶ $\rho : Q \rightarrow B^*$ est la fonction *partielle* finale.

Remarques :

- ▶ On appelle état final un état dans $\text{dom}(\rho)$.
- ▶ L'automate $\langle Q, A, q_0, \delta, \text{dom}(\rho) \rangle$ est déterministe.
- ▶ On étend δ à $Q \times A^*$ comme d'habitude.
- ▶ On étend $\otimes$ à $Q \times A^*$ par
 - ▶ $q \otimes \varepsilon = \varepsilon$ et $q \otimes ua = (q \otimes u) \cdot (\delta(q, u) \otimes a)$

Sémantique des fonctions séquentielles

Définition : Sémantique : $\llbracket \mathcal{A} \rrbracket : A^* \rightarrow B^*$

La sémantique de $\mathcal{A}$ est la fonction *partielle* $\llbracket \mathcal{A} \rrbracket : A^* \rightarrow B^*$ définie par

$$\triangleright \llbracket \mathcal{A} \rrbracket(u) = m \cdot (q_0 \circledast u) \cdot \rho(\delta(q_0, u)).$$

$f : A^* \rightarrow B^*$ est *séquentielle* s'il existe un automate séquentiel $\mathcal{A}$ tel que $f = \llbracket \mathcal{A} \rrbracket$.

Exemples :

1. Transformation d'un texte en majuscules.
2. Codage et décodage avec le code préfixe défini par

$$a \mapsto 0$$

$$b \mapsto 10$$

$$c \mapsto 110$$

$$d \mapsto 111$$

3. Division par 3 d'un entier écrit en binaire en commençant par le bit de poids faible.
4. Multiplication par 3 d'un entier écrit en binaire en commençant par le bit de poids faible.
5. La fonction $f : A^* \rightarrow A^*$ définie par $f(u) = u(ab)^{-1}$.

Propriétés des fonctions séquentielles

Lemme :

Une fonction séquentielle peut être réalisée par un automate séquentiel ayant un préfixe initial vide ($m = \varepsilon$).

Proposition :

Une fonction séquentielle peut être réalisée par un automate **émondé**, i.e., tel que $\forall p \in Q, \exists u, v \in A^*$ tels que $\delta(q_0, u) = p$ et $\delta(p, v) \in \text{dom}(\rho)$.

Proposition :

- ▶ Le domaine d'une fonction séquentielle est un langage reconnaissable.
- ▶ L'image d'une fonction séquentielle est un langage reconnaissable.

Théorème : Image directe, image inverse

Soient $f : A^* \rightarrow B^*$ une fonction séquentielle.

- ▶ Si $L \subseteq A^*$ est rationnel alors $f(L)$ est rationnel.
- ▶ Si $L \subseteq B^*$ est rationnel alors $f^{-1}(L)$ est rationnel.

Fonctions séquentielles pures

Définition : Automates séquentiels purs (Mealy machine)

Un automate séquentiel $\mathcal{A} = \langle Q, A, B, q_0, \delta, \otimes, m, \rho \rangle$ est *pur* si

- ▶ $m = \varepsilon$: le préfixe initial est vide
- ▶ $\rho(q) = \varepsilon$ pour tous $q \in Q$: tous les états sont finaux avec sortie vide.

Une fonction $f : A^* \rightarrow B^*$ est *séquentielle pure* s'il existe un automate séquentiel pur $\mathcal{A}$ avec $f = \llbracket \mathcal{A} \rrbracket$.

Composition

Théorème : Composition

Soient $f : A^* \rightarrow B^*$ et $g : B^* \rightarrow C^*$ deux fonctions partielles.

1. Si f et g sont séquentielles alors $g \circ f : A^* \rightarrow C^*$ est aussi séquentielle.
2. Si f et g sont séquentielles *pures* alors $g \circ f$ est aussi séquentielle *pure*.

Exemple : Multiplication par 5

Dans cet exemple, $A = C = \{0, 1\}$, $B = \{0, 1\}^2$ et les mots représentent des entiers codés en binaire en commençant par le bit de poids faible.

On considère les fonctions séquentielles $f : A^* \rightarrow B^*$ et $g : B^* \rightarrow C^*$ définies par $f(n) = (n, 4n)$, i.e., $f(u) = (u00, 00u)$ et $g(n, m) = n + m$.

La fonction $g \circ f$ code la multiplication par 5.

Construire les automates séquentiels réalisant f et g .

En déduire un automate séquentiel pour $g \circ f$.

Produit en couronne

Définition : Produit en couronne

Soient $\mathcal{A} = \langle Q, A, B, q_0, \delta, \otimes, m, \rho \rangle$ et $\mathcal{B} = \langle Q', B, C, q'_0, \eta, \otimes, m', \rho' \rangle$ deux automates séquentiels.

Le produit en couronne $\mathcal{B} \circ \mathcal{A} = \langle Q'', A, C, q''_0, \zeta, \otimes, m'', \rho'' \rangle$ est défini par

- ▶ $Q'' = Q \times Q'$, $q''_0 = \langle q_0, \eta(q'_0, m) \rangle$ et $m'' = m' \cdot (q'_0 \otimes m)$,
- ▶ $\zeta(\langle p, p' \rangle, a) = \langle \delta(p, a), \eta(p', p \otimes a) \rangle$,
- ▶ $\langle p, p' \rangle \otimes a = p' \otimes (p \otimes a)$,
- ▶ $\rho''(\langle p, p' \rangle) = (p' \otimes \rho(p)) \cdot \rho'(\eta(p', \rho(p)))$.

Lemme : Extension à A^*

Pour tout $u \in A^*$, on a

- ▶ $\zeta(\langle p, p' \rangle, u) = \langle \delta(p, u), \eta(p', p \otimes u) \rangle$,
- ▶ $\langle p, p' \rangle \otimes u = p' \otimes (p \otimes u)$.

Preuve (Composition)

1. Si f et g sont réalisées par $\mathcal{A}$ et $\mathcal{B}$ alors $g \circ f$ est réalisée par $\mathcal{B} \circ \mathcal{A}$.
2. Si $\mathcal{A}$ et $\mathcal{B}$ sont purs alors $\mathcal{B} \circ \mathcal{A}$ est pur.

Plus grand préfixe commun

Définition :

- ▶ Tout sous ensemble $\emptyset \neq X \subseteq B^*$ admet un plus grand préfixe commun, i.e., une borne inférieure pour l'ordre préfixe.
On note cette borne $\bigwedge X$.

Remarque :

1. Soit $u \in B^*$ et $\emptyset \neq X \subseteq B^*$.
 - ▶ $\bigwedge u \cdot X = u \cdot \bigwedge X$
 - ▶ si $u \leq \bigwedge X$ alors $\bigwedge u^{-1} \cdot X = u^{-1} \cdot \bigwedge X$
2. Soit $f : A^* \rightarrow B^*$ une fonction partielle. $\bigwedge f(A^*)$ est défini si $\text{dom}(f) \neq \emptyset$.

Résiduels

Définition : Résiduels

Soit $f : A^* \rightarrow B^*$ une fonction partielle et soit $u \in A^*$.

Le résiduel $f_u : A^* \rightarrow B^*$ est défini par

- ▶ $\text{dom}(f_u) = u^{-1}\text{dom}(f)$ et
- ▶ $f_u(v) = (\bigwedge f(uA^*))^{-1}f(uv)$ pour $uv \in \text{dom}(f)$.

$\bigwedge f(uA^*)$ représente tout ce qu'on peut écrire si on sait que la donnée commence par u . Le résiduel $f_u(v)$ est donc ce qui reste à écrire si la donnée est uv .

Exemples : Calculer les résiduels des fonctions suivantes

- ▶ “quotient” $f : A^* \rightarrow A^*$ définie par $f(w) = w(ab)^{-1}$.
- ▶ “copie” $g : A^* \rightarrow A^*$ définie par $g(w) = ww$.

Normalisation

Exemple :

Donner un automate séquentiel réalisant la fonction $f : A^* \rightarrow A^*$ définie par $f(a^{2^n}b) = (ab)^n a$.

Cet automate devra sortir les lettres du résultat le plus rapidement possible.

Définition : Automate normalisé

Intuitivement, un automate est normalisé s'il écrit son résultat au plus tôt.

Soit $\mathcal{A} = \langle Q, A, B, q_0, \delta, \otimes, m, \rho \rangle$ un automate séquentiel et $p \in Q$.

On définit $\mathcal{A}_p = \langle Q, A, B, p, \delta, \otimes, \varepsilon, \rho \rangle$ et $m_p := \bigwedge \llbracket \mathcal{A}_p \rrbracket(A^*)$ si $\llbracket \mathcal{A}_p \rrbracket(A^*) \neq \emptyset$.

L'automate $\mathcal{A}$ est *normalisé* si pour tout $p \in Q$, $\llbracket \mathcal{A}_p \rrbracket(A^*) \neq \emptyset$ implique $m_p = \varepsilon$.

Proposition : Effectivité

Étant donné un automate séquentiel $\mathcal{A}$, on peut calculer les m_p en temps quadratique (preuve admise).

Normalisation

Proposition : Normalisation

Tout automate séquentiel est équivalent à un automate séquentiel normalisé, qui peut être choisi émondé ou complet.

Preuve

Soit $\mathcal{A} = \langle Q, A, B, q_0, \delta, \otimes, m, \rho \rangle$ un automate séquentiel *émondé* (donc m_p est bien défini pour tous $p \in Q$).

On définit $\mathcal{A}' = \langle Q, A, B, q_0, \delta, \otimes', m', \rho' \rangle$ par :

- ▶ $m' = m m_{q_0} = \bigwedge \llbracket \mathcal{A} \rrbracket (A^*)$,
- ▶ $p \otimes' a = m_p^{-1}((p \otimes a) \cdot m_{\delta(p,a)})$ si $(p, a) \in \text{dom}(\otimes) = \text{dom}(\delta)$
- ▶ $\rho'(p) = m_p^{-1} \rho(p)$ si $p \in \text{dom}(\rho)$

On vérifie que $\mathcal{A}'$ est normalisé et $\llbracket \mathcal{A}' \rrbracket = \llbracket \mathcal{A} \rrbracket$.

Pour obtenir un automate complet, il suffit d'ajouter un état puits.

Résiduels

Lemme : Composition

Soient $u, v \in A^*$. On a $f_{uv} = (f_u)_v$, i.e.,
 $\text{dom}(f_{uv}) = v^{-1}\text{dom}(f_u)$ et $f_{uv}(w) = (\bigwedge f_u(vA^*))^{-1}f_u(vw)$.

Lemme :

Soit $\mathcal{A} = \langle Q, A, B, q_0, \delta, \otimes, m, \rho \rangle$ un automate normalisé complet.
Pour $u \in A^*$, on a $f_u = \llbracket \mathcal{A}_{\delta(q_0, u)} \rrbracket$.

On en déduit qu'une fonction séquentielle réalisée par $\mathcal{A}$ a au plus $|Q|$ résiduels.

Théorème : Caractérisation par résiduels

Une fonction $f : A^* \rightarrow B^*$ est séquentielle si et seulement si elle a un nombre fini de résiduels.

Automate des résiduels

L'automate des résiduels de f est $\mathcal{R} = \langle Q, A, B, q_0, \delta, \otimes, m, \rho \rangle$ où

- ▶ $Q = \{ f_u \mid u \in A^* \}$ (supposé fini pour la réciproque du théorème),
- ▶ $q_0 = f_\varepsilon$ et $m = \bigwedge f(A^*)$ si $\text{dom}(f) \neq \emptyset$, et $m = \varepsilon$ sinon,
- ▶ $\delta(f_u, a) = (f_u)_a = f_{ua}$,
- ▶ $f_u \otimes a = \bigwedge f_u(aA^*)$ si $\text{dom}(f_{ua}) \neq \emptyset$, et $f_u \otimes a = \varepsilon$ sinon,
- ▶ $\rho(f_u) = f_u(\varepsilon)$ si $\varepsilon \in \text{dom}(f_u)$, et $f_u \notin \text{dom}(\rho)$ sinon.

Lemme :

1. Soient $u, v \in A^*$. On a $\delta(f_u, v) = f_{uv}$.
2. Soient $u, v \in A^*$. On a $f_u \otimes v = \bigwedge f_u(vA^*)$ si $\text{dom}(f_{uv}) \neq \emptyset$.
3. Soit $u \in A^*$. On a $f_u = \llbracket \mathcal{R}_{f_u} \rrbracket$.
4. $f = \llbracket \mathcal{R} \rrbracket$.
5. L'automate des résiduels est normalisé, accessible et complet.
6. L'automate des résiduels est minimal parmi les automates normalisés et complets qui réalisent f .